PaySME Payment Modal - API Integration Guide

Version: 1.13.1 - **Updated:** 4 November 2026 **Audience:** Merchants integrating PaySME payments on web, mobile, or server-side.

---

1. Overview

The PaySME Payment Modal is a hosted/embeddable checkout that lets your customers pay using:

1. **PaySME Code** - a short code redeemed at participating Payment Vendors (cash/kiosk/teller).
2. **WayaMe** - merchant-address payment request with bank-app and QR channels.
3. **MTC Maris** - available to approved merchants through the PaySME checkout.
4. **PayPulse** - available to approved merchants through the PaySME checkout.
5. **PayToday** - available to approved merchants through the PaySME checkout.
6. **Card Payment** - processed via **Adumo Online** (Visa / Mastercard).
7. **Lesaka 4All vouchers** - customers redeem a 16-digit voucher PIN using the Lesaka account configured for your business.

Payment options are determined by your merchant configuration and checkout eligibility. Enable Lesaka under **Merchant Portal > API & integration > Facilitator Merchant Codes > Lesaka**; see section 5.6.

Each business can record its three required supplier test cycles in the private Lesaka three-cycle UAT workflow: https://merchant.paysme.site/docs/PaySME_Lesaka_UAT_Workflow.md. Read-only serial enquiries can identify voucher states across the approved inventory without redemption. The staging Lesaka card opens the business test page. It supports automatic API checks, manual portal cases and background recording from the ordinary SDK modal or hosted checkout. Each business uploads its own voucher file. Test checkouts never settle live money. After three successful cycles, exported evidence and Lesaka approval, the card switches that business to Live without changing SDK methods or website configuration.

There are **two ways** to use it:

Method	When to use
PaySME.js SDK	Easiest. Drop a `<script>` in your site and call `PaySME.init`, `create`, `pending`, or the opt-in `requestToPay`.
Direct REST API	Server-to-server processing. It does not authorise replacing the official customer-facing Request-to-Pay button.

The SDK is the recommended integration. It owns the checkout UI and receives centrally managed PaySME updates without requiring merchants to rebuild their websites.

1.1 Choose the SDK behavior explicitly

SDK method	Behavior	Opens payment modal?
`PaySME.init(config)`	Default fixed-amount checkout	**Yes**
`PaySME.create(config)`	Default dynamic checkout	**Yes**, when `.open()` is called
`PaySME.pending(config)`	Opens an already-issued PaySME Code	**Yes**, when `.open()` is called
`PaySME.renderRequestToPayButton(config)`	Renders the official branded button, creates a pending transaction, and sends its hosted payment link by SMS	**No**
`PaySME.requestToPay(config)`	Underlying programmatic Request-to-Pay method	**No**

`init` and `create` remain the normal SDK behavior. The SDK does not detect a mobile app and switch modes automatically. SMS-only Request-to-Pay happens only when the developer explicitly selects that route. Customer-facing integrations must render it through `PaySME.renderRequestToPayButton(...)`.

1.2 Implementation contract - read this first

An implementation is correct only when all of the following are true:

1. Load `https://www.paysme.site/sdk/v1/paysme.js` once per page or application shell.
2. Use the generated snippet from **Dashboard > API & Integration** as the source of merchant-specific values.
3. Treat `on_success` as **PaySME Code generated**, not as payment received.
4. Treat `on_payment_success` and `paysme:payment-success` as paid only when PaySME has verified a real provider/vendor completion.
5. Never unlock goods, credit an account, fulfil an order, or mark an invoice paid from a modal screen, redirect query string, or generated PaySME Code alone.
6. Verify server-side merchant webhooks before fulfilment. Webhook verification is the authoritative path for unattended fulfilment.
7. Never place Lesaka usernames/passwords, Adumo, MTC Maris, PayPulse, PayToday, WayaMe sponsor, Supabase service-role, or webhook-secret credentials in browser code.
8. Any provider simulation-result screen is a demonstration only: it cannot mark a transaction paid or fire payment-success callbacks, SMS messages, or merchant webhooks. Lesaka uses the verified voucher redemption flow in section 5.6.
9. `requestToPay` success means that PaySME created a pending transaction and accepted its payment-link SMS for delivery. It does **not** mean the invoice was paid.
10. Customer-facing Request-to-Pay integrations must use the SDK-rendered **Create Payment Request** button. Do not replace, recolour, rename, or imitate it.
11. After a successful pending request, the customer notification must read exactly **Request to Pay Sent. Check your SMS.** Do not add payment-completion wording.
12. VAT treatment is merchant-level configuration under **Merchant Portal > Tax Settings**. Integrations send the merchant-entered `amount` only; they must not calculate VAT in browser or app code. PaySME snapshots the configured VAT mode and rate when it creates the transaction.

1.3 VAT and amount contract

PaySME supports three explicit merchant tax modes across every paycode route:

Tax setting	Meaning of integration `amount`	Payment modal
Do not charge VAT	Final amount	No VAT line
Prices include VAT	Final amount, already VAT-inclusive	Shows the included VAT portion; adds nothing
Add VAT to prices	Net amount before VAT	Calculates VAT server-side and adds it to the customer total

The same rule applies to SDK snippets, hosted payment links, direct API requests, Request to Pay, subscription uploads, reusable QR products and QR Baskets. Webhook consumers should use the transaction `net_amount`, `vat_amount`, `gross_amount`, `tax_mode`, and `vat_rate` snapshot fields when available. This prevents later changes to merchant settings from changing the accounting meaning of an existing paycode.

1.4 Minimum handoff for a developer or coding agent

Give the implementer these four items:

1. The generated Integration Snippet from the merchant portal.
2. This guide or its downloadable PDF.
3. The page/component where the payment button must appear.
4. The production return URL and server webhook URL, if applicable.

The implementer should not ask the merchant for facilitator credentials. Those are saved by the merchant in the PaySME portal and remain server-side.

1.5 Changes since the v1.10 guide

Release	Integration changes
v1.11.0	Added the voucher payment option, merchant-controlled checkout eligibility, and failed-payment retry routing.
v1.11.1	Updated the Lesaka artwork and lime-green button, and corrected the customer flow to use the receipt's 16-digit voucher PIN.
v1.12.0	Added merchant-specific Lesaka username/password setup, encrypted credential storage, server-verified 4All voucher status and full redemption in NAD, and protected retries for uncertain outcomes. Customer-facing names are Lesaka; the existing `kazang` integration key remains compatible.
v1.13.0	Releases the existing Lesaka checkout contract with updated customer-outcome guidance and the private three-cycle UAT recording workflow. Public method signatures stay compatible.
v1.13.1	Makes voucher digits visible and smaller, unifies black hosted checkout, fixes desktop provider-button overlap, labels completed-payment retries and adds recorder shortcuts for short/long PIN tests.

The public SDK methods and merchant snippet parameters remain compatible. Existing static, dynamic, pending-code and hosted checkouts use the saved merchant configuration automatically. This guide now covers Lesaka setup, checkout requests, payment results, eligibility, errors and retries in addition to the existing VAT and Request-to-Pay contracts.

2. Base URLs

Resource	URL
SDK v1.13.1 (stable v1 URL)	`https://www.paysme.site/sdk/v1/paysme.js`
Hosted Payment Page	`https://paysme.site/pay/:merchant_id?invoice_id=...&amount=...&product_name=...`
API Base	`https://zvoqrqdnuupdefsuiurt.supabase.co/functions/v1`

Endpoints

Endpoint	Purpose
`POST /payments`	Create PaySME Code transactions and supported merchant actions
`POST /mobile-request-to-pay`	Opt-in SMS-only Request-to-Pay for mobile/app integrations
`POST /adumo-card`	Initiate a card payment (returns Adumo JWT + redirect form fields)
`POST /checkout-eligibility`	Resolve enabled payment methods for the merchant and transaction
`POST /mobiwand-payment`	Checkout-managed Lesaka voucher confirmation using the compatible `kazang` provider key; see section 5.6
`POST /webhooks-payments`	Internal - vendor redemption, Adumo notify, merchant webhook fan-out
`GET /adumo-card?action=redirect-success`	Adumo success return URL
`GET /adumo-card?action=redirect-failed`	Adumo failure return URL

---

3. Authentication

Call type	Auth
`create_transaction` (customer-facing paycode)	**Public** - anon key in `Authorization: Bearer <VITE_SUPABASE_PUBLISHABLE_KEY>`
`mobile-request-to-pay`	Merchant integration credentials in body: Merchant ID UUID + PaySME API key
`create_subscription`, `sms_topup`	**Authenticated merchant** - Supabase user JWT
`adumo-card`	Public; identified by `merchant_id` + `api_key` in body
Lesaka confirmation through `mobiwand-payment`	SDK supplies PaySME request headers and the Merchant ID/API key; Lesaka credentials are loaded only by PaySME's backend
SDK calls	SDK injects the anon key automatically

Never put a Supabase `service_role` key, webhook secret, or payment-provider credential in the browser. The generated snippet contains only the PaySME merchant integration identifiers intended for SDK initialization: **vendor_uuid** (the UUID-format Merchant ID) and the PaySME **api_key**. Do not substitute unrelated portal or provider credentials.

3.1 Merchant ID versus Vendor ID

WARNING: Use Merchant ID for `vendor_uuid`; do not use Vendor ID.

Although the SDK parameter is named `vendor_uuid` for backward compatibility, it represents the PaySME **Merchant ID UUID**. Copy this UUID from **Merchant Portal > API & Integration > Your Merchant Details**.

Identifier	Example	Purpose
Merchant ID	`32f44735-f665-4b5f-94ca-19d1a45d2942`	Required by the SDK and PaySME APIs as `vendor_uuid`
Vendor ID	`USV_32F44735`	Human-readable portal/business reference only, unless a separate API field explicitly asks for it

Correct:

```
vendor_uuid: "32f44735-f665-4b5f-94ca-19d1a45d2942"
```

Incorrect:

```
vendor_uuid: "USV_32F44735"
```

---

4. SDK Integration (Recommended)

```
<script defer src="https://www.paysme.site/sdk/v1/paysme.js"></script>
```

Load the SDK once in your page layout or document `<head>`. Do not use `sdk.paysme.site` as the script source and do not create a new script tag every time the customer clicks the payment button. Keep the button disabled until the required `window.PaySME` method exists; after a reasonable timeout, offer the generated hosted checkout URL instead of leaving the interface permanently on "Loading checkout".

4.0 Automatic SDK updates

The stable `/sdk/v1/paysme.js` URL receives compatible v1 updates automatically on the next successful page or WebView reload. The integrating application does not need a code change for every minor v1 release, but it must follow these rules:

1. Use the exact stable URL above. Do not copy the SDK into the application bundle or File Manager under another filename.
2. Do not require an exact minor version such as `PaySME.version === "1.9.1"`. Test for `window.PaySME` and the method the application uses.
3. Do not permanently cache or intercept the stable SDK in a service worker or WebView. Allow normal HTTP revalidation.
4. Reload the page or recreate the WebView after publishing an SDK update; an already-running JavaScript context cannot replace itself.
5. Documentation does not update an application's source code automatically. Developers must still implement new optional methods or breaking changes described in a new guide.

4.0.1 Merchant handoff to a developer or AI agent

Use Merchant ID for `vendor_uuid`; do not use Vendor ID. The Merchant ID is the UUID shown in the API & Integration section. A human-readable value such as `USV_32F44735` is a Vendor ID and must not be placed in `vendor_uuid`.

For the recommended SDK integration, the merchant should provide the developer or AI agent with:

1. The **generated Integration Snippet** copied from **Dashboard > API & Integration**.
2. This API Integration Guide, either as the portal documentation link or the downloadable PDF.
3. The page or component where the **Pay with PaySME** button must appear.
4. The merchant's preferred return page and webhook URL, when server-side payment notifications are required.

The generated snippet contains the merchant-specific PaySME `vendor_uuid` (Merchant ID UUID), `api_key`, invoice settings, amount, and SDK initialization. The developer may adapt the surrounding layout and callback handling, but must preserve those PaySME fields and load the SDK only once. Vendor ID is for portal/business reference only unless a separate API field explicitly requests it.

MTC Maris, PayPulse, PayToday, WayaMe, and Adumo credentials must **not** be copied into the website snippet or given to a browser-side AI tool. The merchant saves the applicable merchant configuration in the PaySME portal; PaySME uses it securely from the backend when the corresponding payment button is enabled.

4.1 Static - fixed amount

```
const payButton = document.getElementById("paysme-pay-btn");

function isPaySMEReady() {
  return !!(window.PaySME && window.PaySME.init);
}

function updatePayButton() {
  payButton.disabled = !isPaySMEReady();
  payButton.textContent = isPaySMEReady() ? "Pay with PaySME" : "Loading checkout...";
}

updatePayButton();
const sdkCheck = setInterval(() => {
  updatePayButton();
  if (isPaySMEReady()) clearInterval(sdkCheck);
}, 250);
setTimeout(() => clearInterval(sdkCheck), 10000);

payButton.addEventListener("click", () => {
  if (!isPaySMEReady()) return;

  localStorage.setItem("paysme_intent", JSON.stringify({
    invoice_id: "INV-2026-001",
    amount_nad: 150,
    redirect_url: window.location.href
  }));

  window.PaySME.init({
    vendor_uuid: "32f44735-f665-4b5f-94ca-19d1a45d2942", // Merchant ID UUID, not Vendor ID
    api_key: "YOUR_API_KEY",
    amount_nad: 150,
    invoice_id: "INV-2026-001",
    product_name: "Annual Membership",
    recurring: false,
    recurring_period: "monthly",
    redirect_url: "https://yoursite.com/payment/callback",
    on_success: (r) => {
      localStorage.removeItem("paysme_intent");
      console.log("Code:", r.generated_code, "TX:", r.transaction_id);
    },
    on_payment_success: (r) => console.log("Paid via", r.provider, r.generated_code),
    on_error: (e) => console.error(e.message || e),
    on_cancel: () => console.log("Closed"),
  });
});

<button id="paysme-pay-btn" disabled>Loading checkout...</button>
```

After PaySME Code generation, the modal remains open on the generated-code screen. The SDK does not automatically redirect after code generation; customers can copy the code, choose another payment option, or close the modal themselves. `redirect_url` remains useful as a return/recovery URL for external payment flows such as card payments.

4.2 Dynamic - customer enters amount

```
const paysme = PaySME.create({
  vendor_uuid: "32f44735-f665-4b5f-94ca-19d1a45d2942", // Merchant ID UUID, not Vendor ID
  api_key:     "YOUR_API_KEY",
  on_success:  (r) => alert("Code: " + r.generated_code),
});
document.getElementById("donate").onclick = () => paysme.open();
```

4.3 Pending - show an already-issued code

```
const p = PaySME.pending({
  vendor_uuid:    "32f44735-f665-4b5f-94ca-19d1a45d2942", // Merchant ID UUID, not Vendor ID
  api_key:        "YOUR_API_KEY",
  generated_code: "1234-5678-9012",
});
p.open();
```

4.4 Mobile Request-to-Pay - required branded button

Use `PaySME.renderRequestToPayButton` when an application must send the normal PaySME hosted payment link by SMS without showing payment methods inside the application. This route is **opt-in**. The SDK renders and controls the official dark **Create Payment Request** button. Customer-facing integrations must not replace, recolour, rename, or imitate this button.

```
<div id="paysme-request-button"></div>
<script>
  window.PaySME.renderRequestToPayButton({
      container: "#paysme-request-button",
      vendor_uuid: "32f44735-f665-4b5f-94ca-19d1a45d2942", // Merchant ID UUID, not Vendor ID
      api_key: "YOUR_API_KEY",
      invoice_id: "INV-2026-001",
      amount_nad: 150,
      mobile: "+264811234567",
      email: "buyer@example.com",
      town: "Windhoek",
      // Optional override. When omitted, the SDK derives a stable key from invoice_id.
      idempotency_key: "mobile-order-INV-2026-001",
      on_request_sent: (r) => console.log(r.message, r.request_id),
      on_error: (error) => console.error(error.message),
  });
</script>
```

After a successful pending request, the SDK shows exactly:

Request to Pay Sent. Check your SMS.

The standard customer-facing integration must leave `show_notification` enabled. `PaySME.requestToPay(config)` remains available as the underlying programmatic method for controlled non-visual integrations, but does not replace the branded-button requirement for customer interfaces. Neither method opens, redirects to, or renders the facilitator-selection modal.

4.4.1 Invoice and PaySME-code reuse

The default SDK behavior mirrors the pending-transaction flow: **Merchant ID +** `invoice_id` **identifies one payment request**. If the customer clicks again an hour, a day, or a week later while that transaction remains pending, PaySME reuses the original transaction and the original PaySME code. It does not create a second code. After a one-minute anti-duplicate window, PaySME may resend the SMS containing that same code.

Use a genuinely different invoice reference for each separate obligation or instalment. For example, `1200_A`, `1200_B`, and `1200_C` produce separate transactions and separate PaySME codes. If `1200_A` is already paid, another call for `1200_A` returns `already_paid` and creates no transaction or SMS.

Parameter	Required	Description
`container`	Yes for `renderRequestToPayButton`	CSS selector or HTML element where the SDK renders the official button

Parameter	Required	Description
`vendor_uuid`	Yes	UUID-format Merchant ID; never the human-readable Vendor ID
`api_key`	Yes	PaySME merchant integration key
`invoice_id`	Yes	Merchant invoice/order reference
`amount_nad`	Yes	Amount in Namibian dollars
`mobile`	Yes	`081`, `083`, `085`, `+26481`, `+26483`, or `+26485` format
`email`	Yes	Valid customer email address
`town`	Yes	Customer town supplied by the application
`idempotency_key`	No	Optional override. When omitted, the SDK derives a stable Merchant ID + invoice key so the same invoice reuses its pending transaction/code.
`show_notification`	No	Defaults to `true`; customer-facing integrations must leave it enabled
`on_request_sent`	No	Called after the SMS provider accepts the hosted-link SMS
`on_already_paid`	No	Called instead of `on_request_sent` when the invoice is already paid
`on_error`	No	Called when validation, transaction creation, or SMS submission fails

Successful result:

```
{
  "ok": true,
  "success": true,
  "request_id": "8e770d2e-886a-4d98-b609-c50aeab5995a",
  "transaction_id": "bbf33acf-7232-4c11-960a-c2c124e1a3c8",
  "status": "payment_request_sent",
  "transaction_status": "pending",
  "sms_sent": true,
  "reused": false,
  "masked_mobile": "+26481••••4567",
  "message": "Request to Pay Sent. Check your SMS."
}
```

The customer opens the SMS link outside the application and chooses an available PaySME payment method on the hosted page. Fulfil only after the signed `payment.paid` merchant webhook; neither `payment_request_sent` nor `sms_sent: true` proves payment.

4.5 Callback and event contract

Signal	Meaning	Safe to fulfil?
`on_success(result)`	A PaySME Code was generated successfully	**No** - transaction is normally still pending
`on_payment_success(result)`	PaySME verified a completed payment during the active checkout	Yes, but server webhook verification is preferred
`paysme:payment-success` browser event	Same verified-payment signal for applications using DOM events	Yes, but server webhook verification is preferred
`on_error(error)`	SDK action failed	No
`on_cancel()`	Customer closed the modal	No
`on_request_sent(result)`	Request-to-Pay SMS accepted; transaction remains pending	**No**

PaySME Code generation returns a payload similar to:

```
on_success({
  transaction_id: "TX_0000001234",
  generated_code: "PSM-ABCD-1234",
```

```
  amount: 150,
  invoice_id: "INV-2026-001"
})
```

A verified payment callback/event contains the provider, generated PaySME Code, status, and available transaction identifiers. Lesaka results use `provider: "kazang"` and `payment_method: "kazang"` for compatibility; display the method to customers as **Lesaka**. Do not manufacture or dispatch this event from merchant code.

```
window.addEventListener("paysme:payment-success", (event) => {
  const payment = event.detail;
  console.log(payment.provider, payment.generated_code, payment.status);
  // Prefer waiting for the signed server webhook before automatic fulfilment.
});
```

4.6 Mobile app and WebView integration

For mobile apps, choose one of these supported patterns:

Pattern	Recommendation
Render `PaySME.renderRequestToPayButton` and let the customer continue from the SMS link	Required Request-to-Pay route when payment selection must remain outside the application
Open the generated PaySME hosted checkout URL in an in-app browser or system browser	**Preferred** - smallest integration surface and independent of WebView script restrictions
Render merchant HTML containing the PaySME SDK inside a JavaScript-enabled WebView	Supported when the application controls WebView permissions, Content Security Policy, caching, and navigation

For Request-to-Pay, the application supplies mobile, email, town, amount, and invoice reference from its own authenticated customer/order data. PaySME validates the fields, generates the pending PaySME transaction and Paycode, sends the normal hosted-link SMS, and returns only the request result. The SDK renders the official button and displays exactly **Request to Pay Sent. Check your SMS.** The application updates the order only after receiving PaySME's signed payment webhook.

App-store responsibility remains with the application publisher. Request-to-Pay is not a mechanism for bypassing Apple or Google rules for digital goods, subscriptions, virtual items, or functionality consumed inside the application. Publishers must use the store-required billing method or an approved regional external-payment programme where applicable.

If the SDK is loaded inside a WebView, the app developer must:

1. Enable JavaScript and HTTPS network access.
2. Allow `https://www.paysme.site` in the app/WebView Content Security Policy and network-security configuration.
3. Use the stable SDK tag:

```
<script defer src="https://www.paysme.site/sdk/v1/paysme.js"></script>
```

1. Wait for the method the app uses, for example `window.PaySME?.renderRequestToPayButton`, before enabling the in-app payment button. Do not block on an exact `PaySME.version` string.
2. After ten seconds, offer the generated hosted checkout URL instead of displaying an indefinite loading state.
3. Reload or recreate the WebView when validating an SDK release. Log `window.PaySME?.version` for diagnostics; the current release is `1.13.1`, but exact minor-version equality must not control readiness.
4. Allow navigation to PaySME, Adumo, and approved banking/deep-link destinations used by the enabled payment methods.
5. Treat messages bridged from JavaScript to native code as UI events only. Signed PaySME server webhooks remain the authoritative payment proof.

Example readiness and hosted-fallback logic:

```
const hostedCheckoutUrl = "YOUR_GENERATED_HOSTED_CHECKOUT_URL";
let sdkReady = false;

const sdkCheck = setInterval(() => {
  sdkReady = Boolean(window.PaySME && window.PaySME.init);
  if (sdkReady) clearInterval(sdkCheck);
}, 250);

setTimeout(() => {
  clearInterval(sdkCheck);
  if (!sdkReady) {
    // Enable a button that opens hostedCheckoutUrl.
  }
}, 10000);
```

---

5. Direct REST API

5.1 Create a paycode transaction

`POST /functions/v1/payments`

```
Authorization: Bearer <VITE_SUPABASE_PUBLISHABLE_KEY>
Content-Type: application/json

{
  "action": "create_transaction",
  "payload": {
    "merchant_id": "32f44735-f665-4b5f-94ca-19d1a45d2942",
    "amount": 150.00,
    "type": "paycode",
    "invoice_id": "INV-2026-001",
    "town": "Windhoek",
    "email": "buyer@example.com",
    "mobile": "+264811234567",
    "subscription_type": null
  }
}
```

Response

```
{
  "ok": true,
  "transaction": {
    "transaction_id": "TX_0000001234",
    "generated_code": "PSM-ABCD-1234",
    "merchant_id": "...",
    "amount": 150,
    "invoice_id": "INV-2026-001",
    "business_name": "Acme Co",
    "paycode_status": "pending",
    "created_at": "2026-06-07T10:00:00Z"
  }
}
```

A PaySME-code SMS is dispatched automatically to the supplied `mobile`.

5.2 Mobile Request-to-Pay endpoint

The SDK method above calls this dedicated endpoint. Direct REST callers must send the same fields and retain one stable idempotency key for each invoice. The SDK handles this automatically when `idempotency_key` is omitted.

The endpoint is a server capability, not an alternative visual brand. Any customer-facing application that initiates this route must still present the official SDK-rendered **Create Payment Request** button and the exact success notice **Request to Pay Sent. Check your SMS.**

`POST /functions/v1/mobile-request-to-pay`

```
Content-Type: application/json
X-Idempotency-Key: mobile-order-INV-2026-001
```

```
{
  "vendor_uuid": "32f44735-f665-4b5f-94ca-19d1a45d2942",
  "api_key": "YOUR_API_KEY",
  "invoice_id": "INV-2026-001",
  "amount_nad": 150,
  "mobile": "+264811234567",
  "email": "buyer@example.com",
  "town": "Windhoek",
  "idempotency_key": "mobile-order-INV-2026-001"
}
```

The endpoint authenticates the Merchant ID/API-key pair, validates and canonicalises customer data, rate-limits abuse, creates one pending transaction, and waits for the SMS service to accept the normal PaySME hosted-link message. Repeating a successful key reuses the original transaction and PaySME code. Calls made within one minute are deduplicated without another SMS; later calls may resend the same code. If the transaction is already paid, the endpoint returns `already_paid` and does not create or send anything. If SMS submission fails, retry the same key so PaySME reuses the existing pending transaction.

The response never means paid. The customer's SMS link opens `https://paysme.site/c/:generated_code`, where the customer chooses from the payment methods enabled for that merchant. The merchant's signed `payment.paid` webhook remains authoritative.

5.3 Initiate a Card Payment (Adumo)

`POST /functions/v1/adumo-card`

```
{
  "merchant_id":   "32f44735-f665-4b5f-94ca-19d1a45d2942",
  "amount":        "150.00",
  "invoice_id":    "INV-2026-001",
  "generated_code":"PSM-ABCD-1234",
  "origin_url":    "https://yoursite.com/checkout",
  "town":          "Windhoek",
  "email":         "buyer@example.com",
  "mobile":        "+264811234567",
  "business_name": "Acme Co"
}
```

Response

```
{
  "ok": true,
  "token": "<adumo-jwt>",
  "mref": "PS_INV-2026-001_1774816563",
  "flow": "CARD",
  "currency_code": "NAD",
  "display_price": "150.00",
  "adumo_url": "https://...adumoonline.com/...",
  "redirect_success_url": "https://zvoqrqdnuupdefsuiurt.supabase.co/functions/v1/adumo-
card?action=redirect-success",
  "redirect_failed_url":  "https://zvoqrqdnuupdefsuiurt.supabase.co/functions/v1/adumo-
card?action=redirect-failed"
}
```

Build a hidden `<form method="POST" action="{adumo_url}">` and POST these fields:

Field	Value
`flow`	`CARD` (opens the card flow directly instead of the payment-options page)
`MerchantID`	(provided by Adumo, configured server-side)
`ApplicationID`	(provided by Adumo)
`MerchantReference`	`mref` from response
`Amount`	`amount` (2dp)
`Token`	`token` from response
`AuthoriseCurrencyCode`	`currency_code` (`NAD`)

Field	Value
DisplayPrice	display_price
txtCurrencyCode	currency_code (NAD)
RedirectSuccessfulURL	redirect_success_url
RedirectFailedURL	redirect_failed_url

The shared Adumo sample `puid` may be used with Adumo's public staging credentials while testing multiple merchant flows. In production, a `puid` enables Virtual 1Click and must belong to the individual cardholder; production checkouts must use a customer-specific value or omit it. The merchant's Adumo Application ID must be provisioned for NAD (or have Adumo MCP enabled) for `AuthoriseCurrencyCode=NAD` to control the actual collected currency. `txtCurrencyCode` and `DisplayPrice` are display fields and do not change settlement currency on their own.

After the Adumo flow completes the user is bounced back through `/adumo-card?action=redirect-success|failed`, which then redirects to your `origin_url` with appended query params:

```
{origin_url}?status=success&mref=...&generated_code=...&invoice_id=...
{origin_url}?status=cancelled&mref=...&error_code=05
```

Note (callback URL pattern): The redirect uses the merchant's own `origin_url`. Examples like `skinnerbek://payment/callback?...` are merchant-specific - your app receives the same query parameters at whatever URL you provided as `origin_url`.

5.4 MTC Maris, PayPulse, and PayToday

Each business applies to the applicable provider separately. After approval, save the merchant code issued directly to your business under **Dashboard > API & Integration > Facilitator Merchant Codes**. Do not place provider credentials or merchant codes in browser code; the PaySME SDK uses the saved configuration securely from the backend.

The SDK automatically uses the pending PaySME transaction amount, Namibian-dollar currency, generated PaySME Code, customer mobile number, and merchant name. Merchants do not need to pass separate provider-specific transaction fields in the integration snippet.

Accepted customer mobile formats are local `081...`, `083...`, and `085...`, or international `+26481...`, `+26483...`, and `+26485...`. Local numbers are converted to canonical `+264...` format before payment initiation. Other prefixes are rejected.

The customer flow is:

1. Customer selects **Pay via MTC Maris**, **Pay via PayPulse**, or **Pay via PayToday**.
2. PaySME shows the provider logo, required-balance warning, and Proceed button.
3. Proceed replaces the warning with a branded input for the short-lived confirmation Paycode sent to the customer's phone.
4. The customer enters the numeric provider Paycode before the three-minute timer expires.
5. Confirm shows a seven-second **Waiting for approval** countdown.
6. While provider credentials are pending, the final circled-check screen displays **PASSED: Simulation Testing Event** and identifies the provider API as awaiting credentials.
7. The testing event does not submit a payment, mark the transaction paid, fire success events, send SMS, or invoke merchant webhooks.

8 Once a provider supplies its API, only a verified provider response may complete the transaction and invoke PaySME success notifications.

Provider availability depends on the merchant's approval and active configuration. PaySME keeps all provider connection details internal to the platform.

5.5 WayaMe / UPI

Each merchant verifies their own WayaMe Payment Address under **Dashboard > API & Integration**. The address, provider merchant code, institution and verification result are stored server-side. PaySME's sponsor partner identity and API credentials are platform-level environment variables and must not be included in merchant code or SDK configuration.

The production checkout will create one WayaMe payment request for the existing PaySME transaction. The same request may expose a bank-app intent, a provider-generated QR payload, or both. Opening an app or displaying/scanning a QR must never mark a payment successful by itself. PaySME must wait for a signed provider notification and may use provider status queries as a fallback before changing the transaction from pending to paid and invoking normal PaySME success notifications.

The current interface exposes **Approve in Bank App** and a QR presentation path. Selecting the button or tapping the QR starts a seven-second waiting state and then displays **PASSED: Simulation Testing Event. WayaMe confirmation API is awaiting provider credentials.** The current QR is a placeholder testing URI: scanning it on another device does not transmit the live merchant, amount, or PaySME transaction request. A real scannable payment request requires the provider's documented QR/deep-link format. The testing event never marks a transaction paid and does not fire callbacks, SMS, or merchant notifications.

5.6 Lesaka 4All voucher payments

Merchant setup

Each merchant uses the Lesaka API username and password issued to their own business. Save them under **Merchant Portal > API & integration > Facilitator Merchant Codes > Lesaka**:

Field	Required	Purpose
Lesaka API username	Yes	Authenticates your business's Lesaka account
Lesaka API password	Yes	Authenticates the same Lesaka account
Store ID	No	Optional store metadata when supplied by Lesaka; not a replacement for either credential

Save both required fields together and enable **Show this button to customers**. Lesaka appears when the merchant's saved setup, payment-method visibility, subscription and account standing allow it. An old Kazang merchant code alone does not configure a Lesaka account.

PaySME encrypts both credentials before database storage and loads them only on the backend for the merchant associated with the PaySME transaction. Saved usernames and passwords are not returned to the portal, SDK or hosted checkout. Leave the fields empty unless replacing the saved credentials; enter both again to replace them. Credentials cannot be replaced while voucher attempts need resolution.

Continue using the **PaySME Merchant ID UUID** as `vendor_uuid` and your **PaySME API key** as `api_key` in the generated snippet. Do not substitute a Lesaka username or Store ID. A merchant's identity comes from the PaySME transaction; Store ID is optional metadata, not an instruction to credit a different merchant.

SDK and hosted checkout

Hosted `/pay/` links and existing-code `/c/` links share the black secure-checkout design on desktop and mobile. Selecting a provider hides the underlying payment buttons so they cannot appear beneath its form.

There is no separate Lesaka SDK method or additional credential parameter. `PaySME.init(config)`, `PaySME.create(config)` and `PaySME.pending(config)` offer **Pay via Lesaka** when it is available. Hosted payment links use the same saved merchant setup. In Request-to-Pay mode, the customer can select Lesaka on the hosted page opened from the SMS link.

1. The customer selects **Pay via Lesaka** and uses a Lesaka 4All voucher from a participating Kazang retailer.
2. The voucher's full balance must equal the checkout's final payable amount, including any VAT already calculated by PaySME. Currency is **NAD** (N$). Partial redemption and vouchers with a different balance are not accepted by this checkout.
3. The customer enters the **16-digit PIN from the receipt** into the visible voucher field and selects **Confirm payment**. Entered digits remain readable in the SDK and hosted checkout, with smaller type and spacing to fit the full PIN on mobile. The **12-digit serial number is for status enquiries, not payment redemption**. This is a voucher PIN, not an SMS code or a PaySME Code.
4. PaySME checks that the voucher is Active, unexpired and matches the currency and amount. It then redeems the full balance and verifies the resulting receipt and zero remaining balance.
5. Only verified redemption and matching PaySME settlement produce **Payment confirmed**, `status: "paid"`, the normal payment-success signal and merchant payment notifications.

Opening the modal, entering a PIN or receiving a PaySME Code does not prove payment. Keep the existing `on_success` code-generation behavior; use the verified-payment callback/event for the customer interface and the signed `payment.paid` webhook for unattended fulfilment.

After a successful redemption, retrying the same voucher on the same checkout returns the original result with `replayed: true` and displays **Already completed**. It does not redeem again. A different voucher on that completed checkout requires a new checkout; this is distinct from an unused checkout rejecting a wrong value, consumed, expired or cancelled voucher. The test recorder retains rejections and repeated requests, and repeated completed results do not create new passed cases. For malformed-PIN supplier tests use **Test too-short PIN (15 digits)** or **Test too-long PIN (17 digits)** in the recorder. These send controlled invalid requests and record the responses while ordinary checkout continues to require exactly 16 digits.

Checkout request and compatible identifiers

The SDK submits this PaySME request automatically. Existing integrations do not need a custom voucher form or direct Lesaka API calls:

```
POST /mobiwand-payment
Content-Type: application/json

{
  "action": "confirm_provider_code",
  "provider": "kazang",
  "merchant_id": "YOUR_PAYSME_MERCHANT_UUID",
  "api_key": "YOUR_PAYSME_API_KEY",
  "generated_code": "EXISTING_PAYSME_CODE",
  "confirmation_code": "CUSTOMER_16_DIGIT_VOUCHER_PIN"
}
```

`kazang` remains the stored facilitator/provider/payment-method key for compatibility. It identifies Lesaka in `checkout-eligibility` as `methods.kazang`, in payment responses and in existing integrations; do not rename the request value to `lesaka`. The SDK supplies PaySME authentication headers. Hosted checkout uses PaySME's hosted authorization flow.

PaySME obtains the amount, merchant credentials and redemption reference from the stored transaction and backend records. The browser does not choose them. On the server, PaySME calls Lesaka's `POST /api/v2/voucher/status` and `POST /api/v2/voucher/redeem`; provider authentication stays between PaySME and Lesaka.

Errors and retries

If the connection times out or the outcome is uncertain, retain the **same PaySME checkout/code and the same voucher PIN**. PaySME retains the original redemption reference and protects against duplicate redemption. Do not create a new invoice, change merchant credentials or switch vouchers to work around an unresolved attempt.

For an invalid, inactive, expired or mismatched voucher, follow the displayed error. For an uncertain result, retry the same voucher on the same checkout. If PaySME requests support review or reconciliation, stop submitting vouchers and contact support with the PaySME Code and any returned `traceId`; do not send PINs or passwords.

6. Merchant Webhooks

Configure in **Dashboard -> Profile -> Webhook URL**.

When PaySME verifies a payment completion and the transaction changes to `paid`, PaySME POSTs:

```
POST {your_webhook_url}
Content-Type: application/json
X-PaySME-Signature: sha256=<hex>

{
  "notification_only": true,
  "event": "payment.paid",
  "generated_code": "PSM-ABCD-1234",
  "transaction_id": "TX_0000001234",
  "invoice_id": "INV-2026-001",
  "amount": 150,
  "currency": "NAD",
  "status": "paid",
  "paid_at": "2026-06-07T10:14:22Z",
  "merchant_id": "32f44735-f665-4b5f-94ca-19d1a45d2942"
}
```

Verify signature: HMAC-SHA256 of raw body using your **Webhook Secret** (visible once when generated).

```
const expected = crypto.createHmac("sha256", WEBHOOK_SECRET)
                       .update(rawBody).digest("hex");
// Compare with header value after the "sha256=" prefix.
```

Respond `200` within 10s or the delivery is retried (exponential backoff, up to 6 attempts).

---

7. Transaction Lifecycle

```
created (pending)
   |
   +-- customer pays by card  ---> Adumo notify  ---> paid / finalized
   |
   +-- customer pays by MTC Maris / PayPulse / PayToday ---> verified provider confirmation ---> paid /
finalized
   |
   +-- customer pays by WayaMe ---> signed notify/status verification ---> paid / finalized
   |
   +-- customer pays by Lesaka ---> verified 4All redemption and settlement ---> paid / finalized
   |
   +-- customer redeems code at vendor ---> webhooks-payments ---> paid
                                                          |
                                                          +--> merchant webhook fired
```

paycode_status	Meaning
pending	Code generated, not yet redeemed/paid
paid	Completion verified through vendor redemption or a supported payment provider
expired	Code lifetime elapsed
cancelled	Cancelled by merchant or failed card auth

8. Error Codes

HTTP	`error`	Meaning
400	`Missing action`	Body missing `action` field
400	`Unknown action`	Action not recognised
401	`Unauthorized: missing auth header`	No `Authorization` header on protected action
401	`Unauthorized: invalid token`	Bad/expired user JWT
403	`Forbidden: merchant mismatch`	JWT `sub` != `user_id` in payload
400	`Merchant not found`	`merchant_id` does not exist
400	`Card payment initiation failed`	Adumo JWT generation failed
400	`A valid customer email address is required`	Request-to-Pay email validation failed
400	`Use a Namibian mobile number...`	Request-to-Pay mobile prefix/length validation failed
401	`Invalid merchant credentials`	Request-to-Pay Merchant ID/API-key pair is invalid
409	`This payment request is already being processed`	Duplicate idempotency key is still processing
409	`This idempotency_key is already assigned...`	Same key was reused with different payment details
429	`Too many payment requests`	Merchant/customer SMS rate limit reached
502	`The payment was created, but the SMS could not be sent`	Retry with the same idempotency key

Lesaka voucher errors return a readable `error` and, where available, a machine-readable `code`:

HTTP	`code`	Handling
400	`invalid_voucher_pin`	Enter the 16-digit voucher PIN
409	`voucher_mismatch`	Voucher balance/currency must match the final payable NAD amount
409	`voucher_not_active`	The displayed message identifies an already used, expired, cancelled or reversed voucher; use another voucher
502	`lesaka_outcome_unknown`	Retry the same voucher on the same checkout
409	`lesaka_account_changed`	Contact support before retrying with changed merchant credentials
409	`lesaka_reconciliation_required`	Do not submit another voucher; ask support to reconcile the payment
503	`lesaka_not_configured`	Merchant/platform configuration is unavailable; choose an available method

The existing SDK and hosted checkout display the backend customer message. Known used, expired, cancelled and reversed states are identified before balance matching, so a zero balance is not presented as a checkout value error. Unsupported vouchers ask the customer to use a supported voucher; merchant/provider authorization problems ask them to choose another payment method or contact the merchant. An uncertain outcome instructs them to retry the same voucher on the same checkout. SDK v1.13.1 keeps the same credential-free public methods. Existing compatible SDK integrations also receive these centrally served customer messages.

Provider-reported failures may return a `lesaka_<number>` code and a `trace_id` (the provider calls it `traceId`). Use the displayed message and retain those identifiers for support; a failed or uncertain response is not proof of payment.

Adumo card error codes (returned in `error_code`):

Code	Meaning
`00`	Approved
`05`	Do not honour / declined
`51`	Insufficient funds
`54`	Expired card
`91`	Issuer unavailable
`96`	System malfunction

---

9. Testing

Type	Value
Test Merchant ID	`00000000-1986-0026-0000-000000000001`
Test Card (approve)	`4111 1111 1111 1111`, any future expiry, CVV `123`
Test Card (decline)	`4000 0000 0000 0002`

Run the SDK demo: https://www.paysme.site/sdk/v1/demo.html

SDK v1.13.1 includes merchant-specific Lesaka 4All voucher payments and retains merchant VAT lookup, the Namibia VAT breakdown and the official `renderRequestToPayButton` route. Existing `init`, `create` and `pending` integrations keep their behavior and use the saved Lesaka setup automatically. The stable v1 SDK URL receives compatible updates after reload and HTTP revalidation. Log `window.PaySME.version` and check the required method exists; do not reject a compatible release solely because its minor version changed. The card examples above are card-only data and must not be used as Lesaka credentials or voucher PINs.

---

10. Going Live Checklist

- [ ] Production `merchant_id` & `api_key` in your code/env
- [ ] Webhook URL registered + secret stored server-side
- [ ] Signature verification implemented
- [ ] `origin_url` set to your production return page
- [ ] KYC approved (Dashboard -> KYC)
- [ ] SMS credits topped up
- [ ] Modal tested on mobile + desktop
- [ ] Card flow tested end-to-end with live amount of N$ 1.00
- [ ] `on_success` does not fulfil an order or mark an invoice paid
- [ ] Signed `payment.paid` webhook is verified before unattended fulfilment
- [ ] If offering Lesaka, the merchant's own username/password are saved in the portal and its customer-facing button is enabled

- [ ] Lesaka success requires verified full-value NAD redemption and settlement; an uncertain attempt retains the same voucher and checkout
- [ ] Provider testing-result screens leave the PaySME transaction pending
- [ ] If using Request-to-Pay, the app uses the SDK-rendered branded button and displays exactly Request to Pay Sent. Check your SMS.
- [ ] Request-to-Pay never marks the invoice paid from the request result
- [ ] A stable idempotency key is retained for retries of the same mobile payment request
- [ ] App-store billing rules have been reviewed for the exact goods/services sold by the application

10.1 Coding-agent acceptance test

Before handing the integration back, a developer or coding agent should report evidence for each item:

1. SDK script is loaded once from the stable `www.paysme.site/sdk/v1/paysme.js` URL, reports the current release for diagnostics, and exposes the required method without an exact minor-version gate.
2. Generated merchant `vendor_uuid` contains the UUID-format Merchant ID (never the human-readable Vendor ID) and the PaySME `api_key` is preserved.
3. Static or dynamic modal opens from the intended button.
4. Town, email, and accepted Namibian mobile formats validate correctly.
5. PaySME Code generation returns `on_success` but does not trigger fulfilment.
6. Closing the modal invokes only `on_cancel`.
7. Provider simulation flows, where present, do not fire `on_payment_success`. Lesaka instead requires verified redemption and settlement, returns the compatible `kazang` key, and preserves the same voucher/checkout for uncertain retries.
8. Server webhook verification uses the raw request body and webhook secret.
9. No provider credentials, webhook secrets, or service-role keys appear in browser source, logs, screenshots, or commits.
10. If the application selected Request-to-Pay, the SDK-rendered branded button is present, no facilitator/payment modal opens inside the app, the exact SMS-check confirmation is shown, and fulfilment still waits for the signed webhook.

---

11. Support

- Docs: https://paysme.site/api
- Email: support@paysme.site
- Status / Logs: Dashboard -> API Integration -> Logs